

Mesa — Full-Stack Project Report

Mesa — Restaurant Table Reservation Platform

Student Name: Asan Limbu **Target Audience:** Diners booking tables at independent UK restaurants, and the managers who run those venues.

Project Summary & Problem Statement

Independent restaurants still take bookings by phone and paper diary. The results are routine: double-booked tables, parties seated at tables too small for them, unrecorded no-shows, and a process that forces customers to call during service. Diners, meanwhile, have nowhere to see which nearby restaurants genuinely have a table free at the time they want to eat.

Mesa is a multi-restaurant reservation platform. Diners search by cuisine, city, price and real table availability, then book. Managers administer their own venue: its tables, the reservation queue, and reporting on covers, occupancy and no-shows.

Core Features & User Stories

1. **Authentication and ownership-scoped access control.** JWTs with bcrypt-hashed passwords, and separate `authenticate` (401) and `authorize` (403) middleware. Role alone is deliberately insufficient: a manager may act only on the venue they own, and cross-venue requests return 404 rather than 403, so the API never confirms records the caller cannot see.
2. **Restaurant discovery.** Search by name; filter by cuisine, city, price band and genuine availability for a given date, time and party size. Sortable and paginated.
3. **Reservation engine.** Table-level conflict detection that makes double-booking impossible by construction.
4. **Manager dashboard.** Covers today, occupancy, cancellation and no-show rates, a 30-day series and busiest service hours.
5. **Diner reservation management.** View, modify and cancel your own bookings.

Technical Stack

React 19 with Vite; Context and hooks for the only genuinely global state. Tailwind for a bespoke visual system. Node.js with Express 5. Prisma ORM over SQLite, portable to PostgreSQL unchanged. Reservations are inherently relational — availability joins restaurant, table and reservation — so a document store would push those joins into application code. The brief requests a “Django ORM” on a Node back end, which is impossible; Prisma satisfies that intent.

System Architecture & Process Logic

The React SPA sends JSON over HTTP with a bearer token. Express resolves routes through auth middleware into a service layer, which reaches the database only through Prisma. Booking logic sits beneath the services as pure functions with no I/O. Once built, the same server also serves the compiled front end.

Booking a table: the client POSTs the sitting; the server validates it, opens a transaction, loads tables large enough for the party, filters them through the conflict engine, and allocates the smallest survivor. Re-checking inside the transaction closes the race on the last table. When nothing is free the API answers 409 with nearby available times.

Challenges & Ethical Considerations

The difficulty was correctness, not features. Half-open intervals let a table freed at 20:00 be rebooked at 20:00; services running past midnight defeat naive time comparison; and deriving dates with `toISOString` shifted every dashboard label a day early under British Summer Time. Because the engine is pure, 65 tests cover these and the API directly. Holding bookings means holding personal data, so passwords are hashed and login errors are identical to prevent account enumeration. Future work: payments, notifications, deployment.